

REST API CLIENT

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	1
Wymagania.....	2
Badanie API	2
Implementacja	2
Commit projektu do GIT.....	7
Podsumowanie.....	8

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- pobieranie danych z zewnętrznych zasobów za pomocą REST API
- zdobywanie wiedzy na temat zewnętrznych API za pomocą dokumentacji typu Swagger
- wysyłanie asynchronicznych żądań z wykorzystaniem XMLHttpRequest i Fetch API

W praktycznym wymiarze uczestnicy stworzą dynamiczną stronę HTML pozwalającą na wyświetlanie bieżącej informacji pogodowej oraz prognoz dla zadanej przez użytkownika miejscowości.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie wykonywania połączeń synchronicznych i asynchronicznych z poziomu JS na stornie.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

WYMAGANIA

W ramach LAB D przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS
- pole tekstowe (input typu „text”) do wprowadzania adresu
- przycisk „Pogoda”, po kliknięciu którego wykonywane jest zapytanie asynchroniczne:
 - do API Current Weather: <https://openweathermap.org/current> za pomocą XMLHttpRequest
 - do API 5 day forecast: <https://openweathermap.org/forecast5> za pomocą Fetch API
- obsługa zwrotki z obu API – wypisanie pogody bieżącej oraz prognoz poniżej pola wyszukiwania.

Wygeneruj klucz do API. Ponieważ aktywacja może chwilę potrwać, na czas trwania laboratorium możesz wykorzystać „służbowy” klucz: `7ded80d91f2b280ec979100cc8bbba94`. **UWAGA!** Klucz zostanie dezaktywowany niedługo po zajęciach. Musisz wygenerować swój własny.

W przypadku blokady twórczej można posiłkować się filmem: <https://www.youtube.com/watch?v=WoKp2qDFxKk> jednakże spróbuj rozwiązać ten problem samodzielnie!

Prowadzący omówi powyższe wymagania. Upewnij się, czy wszystko rozumiesz.

Tu umieść swoje notatki:

...notatki...

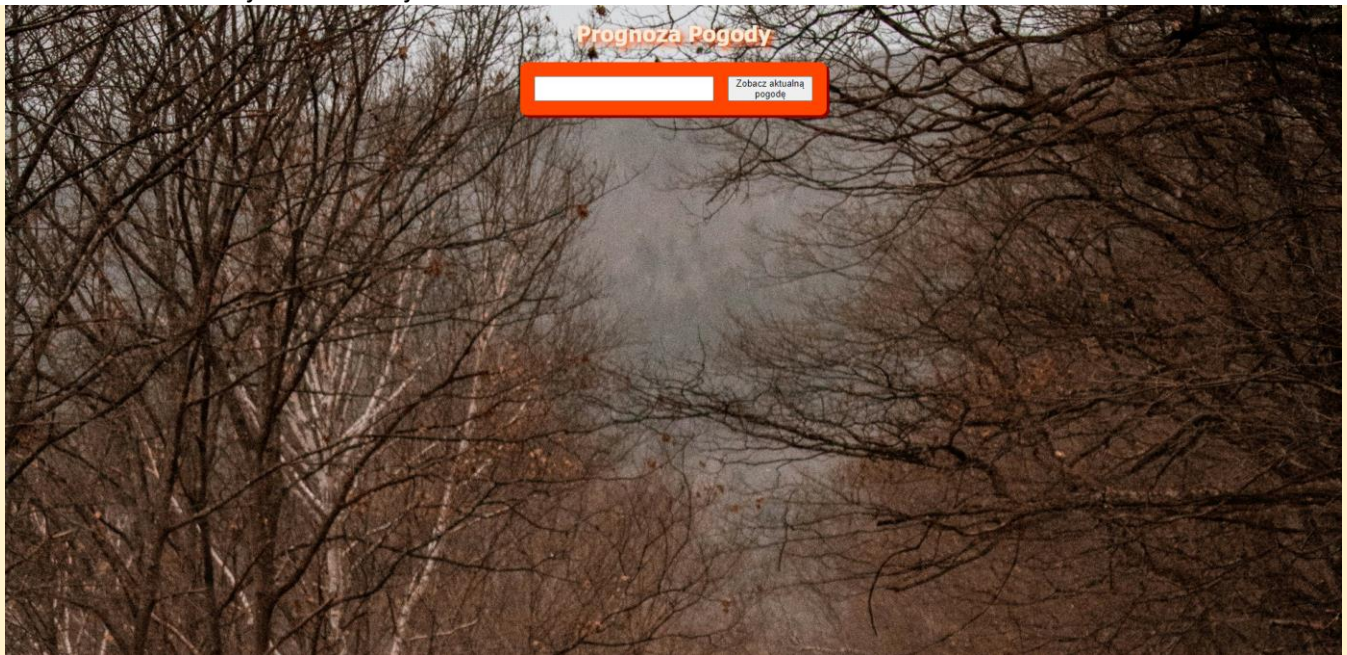
BADANIE API

Poświęć kilka minut na wykonanie przykładowych zapytań do API z poziomu pasku adresu przeglądarki. Podaj wymagane parametry dla osiągnięcia różnych wyników. Zbadaj odpowiedzi API, aby uzyskać pełen obraz wymagań i możliwości API.

IMPLEMENTACJA

Tradycyjnie implementację należy zacząć od zbudowania w HTML + CSS wszystkich wymaganych elementów / placeholderów na te elementy. Następnie krok po kroku należy implementować poszczególne zachowania.

Wstaw zrzut ekranu zawierającego stronę ze wszystkimi elementami, tj. pole tekstowe, przycisk, miejsce do wyświetlenia pogody i prognozy:



Punkty:

0

1

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do current za pomocą XMLHttpRequest:

```
getCurrentWeather(query) :void {
  let url :string = this.currentWeatherLink.replace( {searchValue: "{query}", query});
  console.log(url);

  let req :XMLHttpRequest = new XMLHttpRequest();
  req.open( method: "GET", url, async: true);
  req.addEventListener( type: "load", listener: () :void => {
    console.log(JSON.parse(req.responseText));
    this.currentWeather = JSON.parse(req.responseText);
    this.drawWeather();
  });
  req.send();
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

```
▼ {coord: {...}, weather: Array(1), base: 'stations', main: {...}, visibility: 10000, ...} pogodynka.js:39
  base: "stations"
  ▶ clouds: {all: 97}
  cod: 200
  ▶ coord: {lon: 14.553, lat: 53.4289}
  dt: 1732669424
  id: 3083829
  ▶ main: {temp: 5.35, feels_like: 3.54, temp_min: 4.32, temp_max: 6.53, pressure: 1019, ...}
  name: "Szczecin"
  ▶ rain: {1h: 0.11}
  ▶ sys: {type: 2, id: 2034200, country: 'PL', sunrise: 1732690128, sunset: 1732719037}
  timezone: 3600
  visibility: 10000
  ▼ weather: Array(1)
    ▶ 0: {id: 500, main: 'Rain', description: 'słabe opady deszczu', icon: '10n'}
      length: 1
    ▶ [[Prototype]]: Array(0)
  ▶ wind: {speed: 2.24, deg: 250, gust: 4.02}
  ▶ [[Prototype]]: Object
```

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do forecast za pomocą Fetch:

```

getForecast(query) :void {
  let url :string = this.forecastLink.replace( {searchValue: "{query}"}, query);
  fetch(url) Promise<Response>
    .then((response :Response ) => {
      return response.json();
    }) Promise<any>
    .then((data) :void => {
      console.log(data);
      const hourlyForecast :any[] = this.interpolateForecast(data.list);
      this.forecast = hourlyForecast;
      this.drawWeather();
    })
  ;
}

```

```

//funkcja interpolacji, która zmienia podziałkę prognozy na co godzinę
Show usages  Wojtuch566
interpolateForecast(forecastData) :any[] {
  const interpolatedData :any[] = [];

  for (let i :number = 0; i < forecastData.length - 1; i++) {
    const current = forecastData[i];
    const next = forecastData[i + 1];

    //odsep czasu między punktami w sekundach
    const currentTime = current.dt;
    const nextTime = next.dt;
    const timeDiff :number = (nextTime - currentTime) / 3600; //w godzinach

    //dodawanie bieżącego punktu
    interpolatedData.push(current);

    //interpolowanie brakujących godzin
    for (let t :number = 1; t < timeDiff; t++) {
      const factor :number = t / timeDiff;
      interpolatedData.push({
        dt: currentTime + t * 3600,
        main: {
          temp: current.main.temp + factor * (next.main.temp - current.main.temp),
          feels_like: current.main.feels_like + factor * (next.main.feels_like -
            current.main.feels_like)
        },
        weather: current.weather
      });
    }
  }

  //dodawanie ostatniego punktu
  interpolatedData.push(forecastData[forecastData.length - 1]);

  return interpolatedData;
}

```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

```

▼ {cod: '200', message: 0, cnt: 40, list: Array(40), city: {}} pogodynka.js:53
  ► city: {id: 3083829, name: 'Szczecin', coord: {_, _}, country: 'PL', population: 407811, _}
    cnt: 40
    cod: "200"
  ▼ list: Array(40)
    ► 0: {dt: 1732676400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 1: {dt: 1732687200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 2: {dt: 1732698000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 3: {dt: 1732708800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 4: {dt: 1732719600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 5: {dt: 1732730400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 6: {dt: 1732741200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 7: {dt: 1732752000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 8: {dt: 1732762800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 9: {dt: 1732773600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 10: {dt: 1732784400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 11: {dt: 1732795200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 12: {dt: 1732806000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 13: {dt: 1732816800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 14: {dt: 1732827600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 15: {dt: 1732838400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 16: {dt: 1732849200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 17: {dt: 1732860000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 18: {dt: 1732870800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 19: {dt: 1732881600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 20: {dt: 1732892400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 21: {dt: 1732903200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 22: {dt: 1732914000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 23: {dt: 1732924800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 24: {dt: 1732935600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 25: {dt: 1732946400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 26: {dt: 1732957200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 27: {dt: 1732968000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 28: {dt: 1732978800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 29: {dt: 1732989600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 30: {dt: 1733000400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 31: {dt: 1733011200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 32: {dt: 1733022000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 33: {dt: 1733032800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 34: {dt: 1733043600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 35: {dt: 1733054400, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 36: {dt: 1733065200, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 37: {dt: 1733076000, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 38: {dt: 1733086800, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    ► 39: {dt: 1733097600, main: {_, _}, weather: Array(1), clouds: {_, _}, wind: {_, _}}
    length: 40
    ► [[Prototype]]: Array(0)
    message: 0
    ► [[Prototype]]: Object

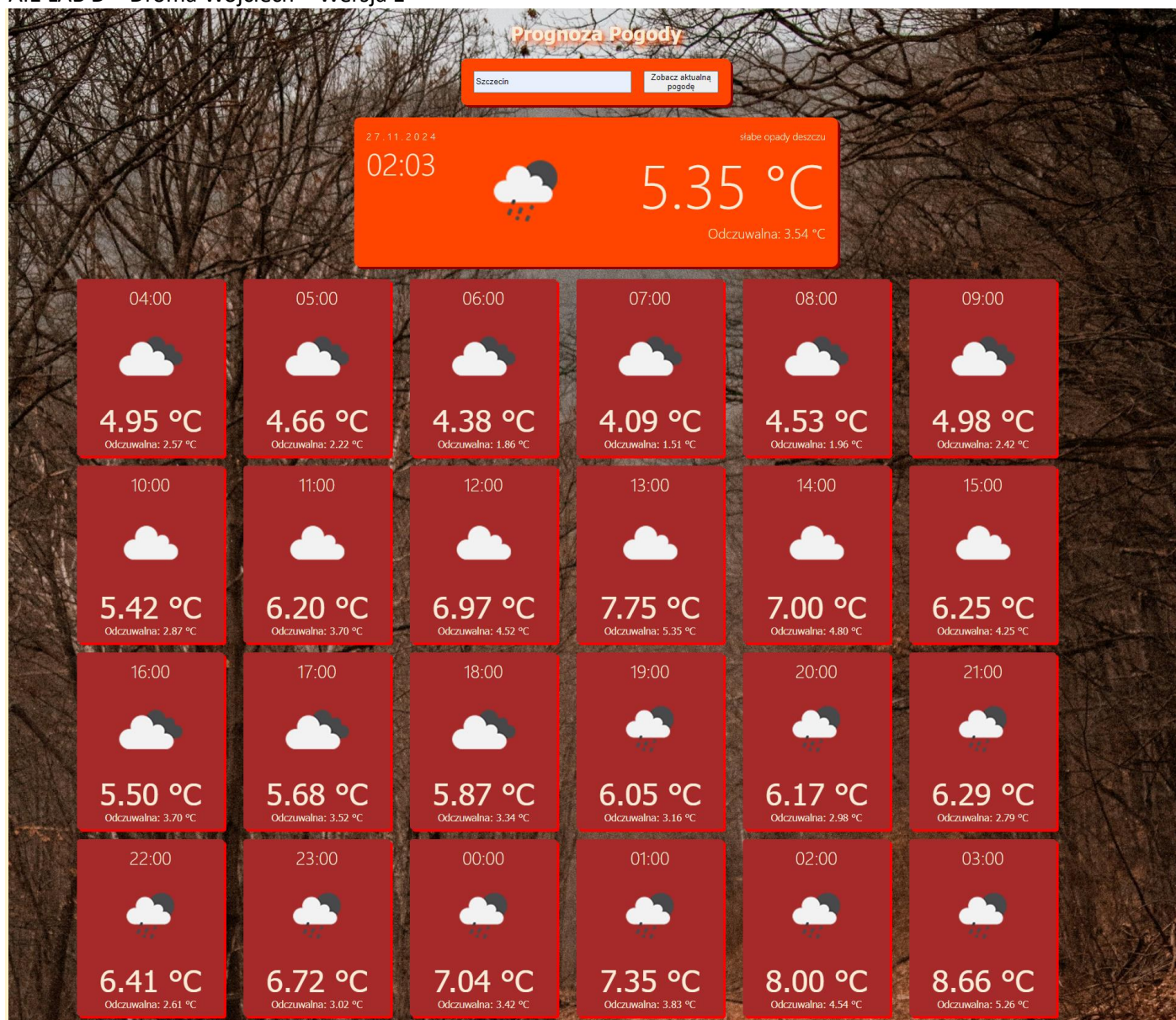
```

Punkty:

0

1

Wstaw zrzut ekranu przedstawiającego wizualizację prognoz pogody:



Upewnij się, że widoczne są pasek wyszukiwania ze wskazaną miejscowością, a także zarówno pogoda bieżąca jak i prognozy pogody.

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-d` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-d` w swoim repozytorium:

<https://github.com/Wojtuch566/Aplikacje-Internetowe-1/tree/lab-d>

...link, np. <https://github.com/inazwisko/ai1-lab/tree/lab-d...>

PODSUMOWANIE

W kilku zdaniach podsumuj zdobyte podczas tego laboratorium umiejętności.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.